

Melhores Práticas de Arquitetura

Marco Mendes

Padrões Arquiteturais

Marco Mendes

Arquitetura de software não é arte.

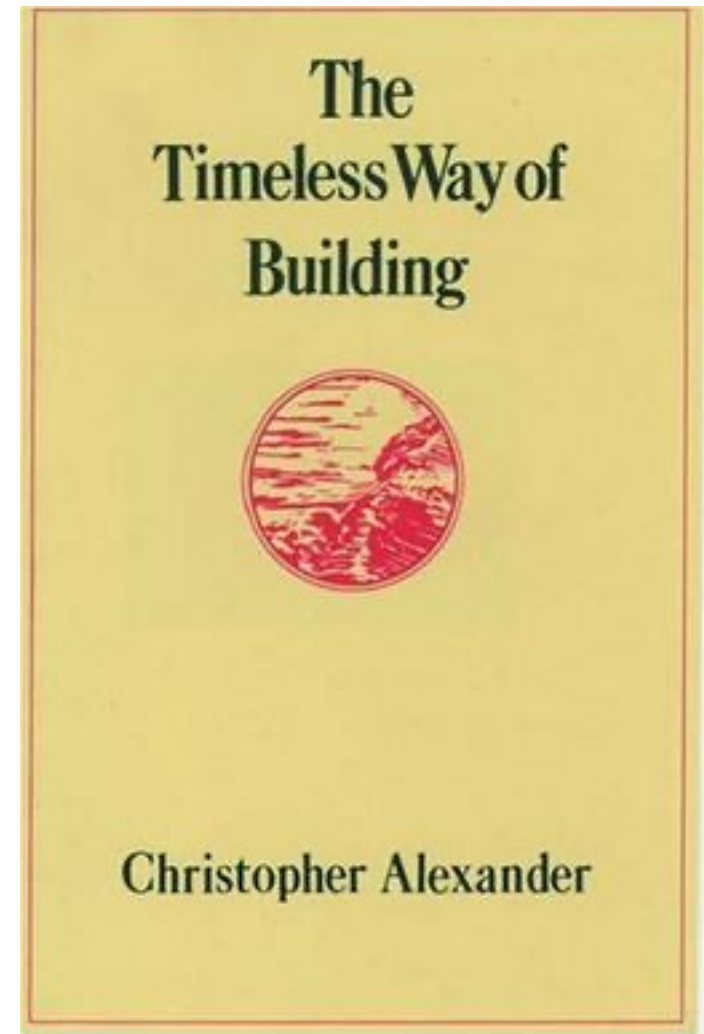
- ❖ O arquiteto de software não é um inventor.
- ❖ A criatividade do arquiteto é direcionada para a busca eficaz de soluções.
- ❖ O arquiteto soluciona grandes problemas combinando soluções já encontradas.

O pragmatismo na arquitetura de software

- ❖ Reinventar a roda ou concluir o projeto?
- ❖ Resultados são essenciais para sua credibilidade e para as empresas.
- ❖ Arquitetos devem ser produtivos.

Padrões

Práticas reusáveis
provadas em projetos
e produtos.



The
Timeless Way of
Building



Christopher Alexander

Definição de padrão “*pattern*”

“Cada padrão descreve um problema no nosso ambiente e o núcleo da sua solução, de forma que você possa usar esta solução mais de um milhão de vezes, sem nunca fazê-lo da mesma maneira.”

– Christopher Alexander



“As melhores arquiteturas
estão repletas de padrões.”
– Grady Booch

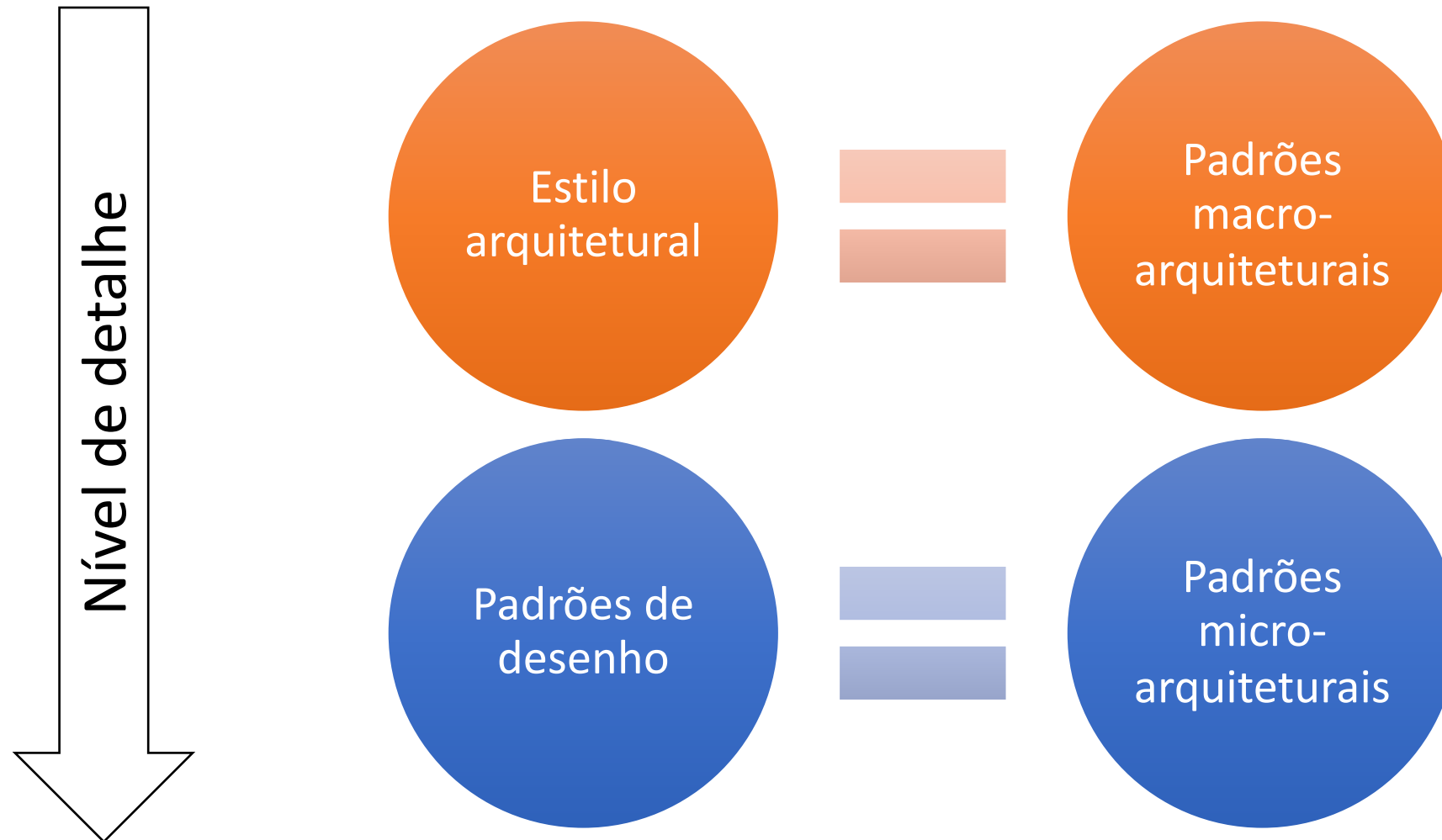
Padrões na Arquitetura de Software

Ferramenta para produtividade intelectual

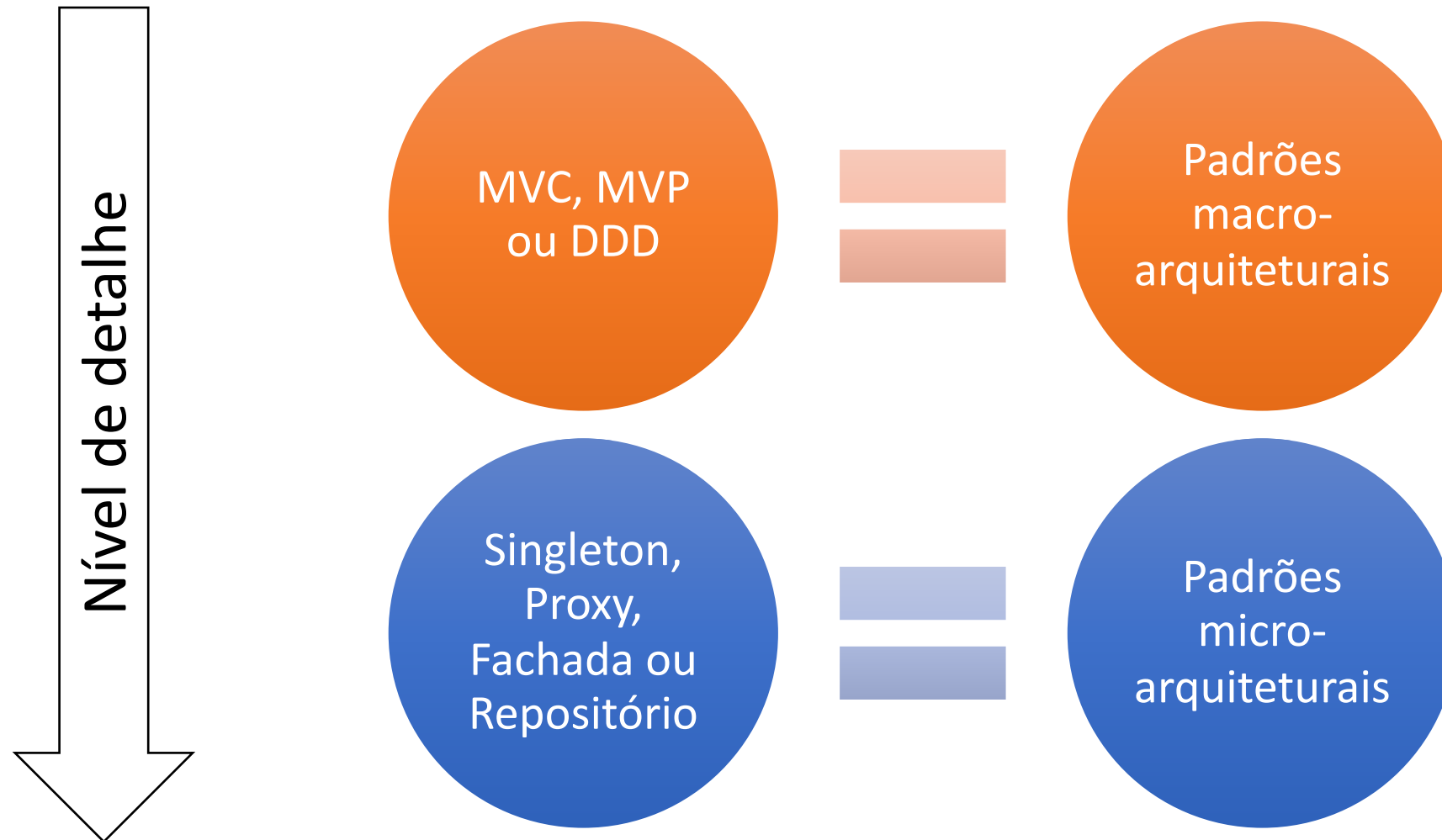
- Para se tornar um “Faixa Preta” em arquitetura de software
 - Estude desenhos de outros mestres
 - Colete os padrões
 - Memorize os padrões de arquitetura
 - Aplique repetidamente conforme o contexto



Padrões comuns na arquitetura de software



Padrões comuns na arquitetura de software

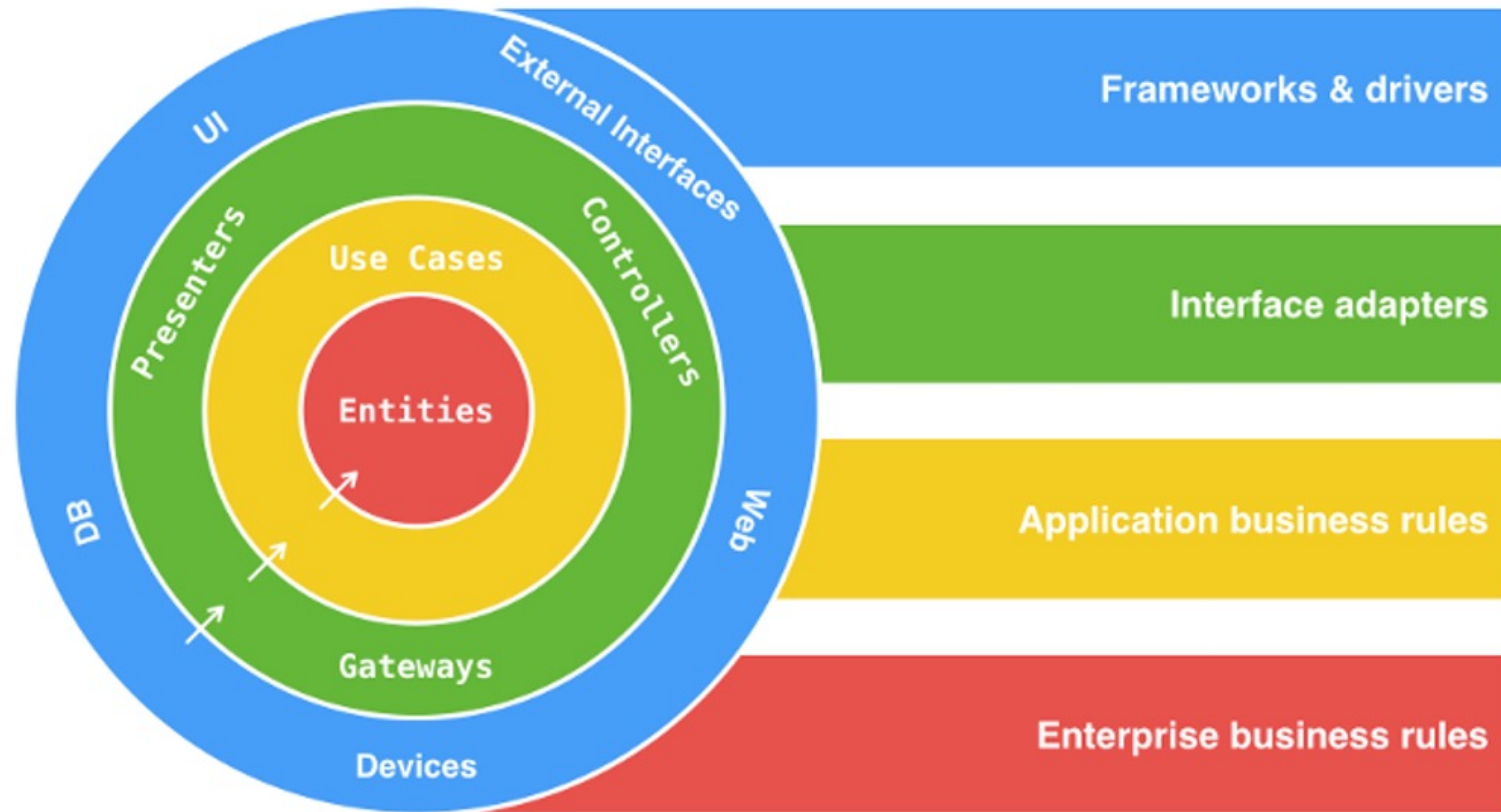


Padrões arquiteturais

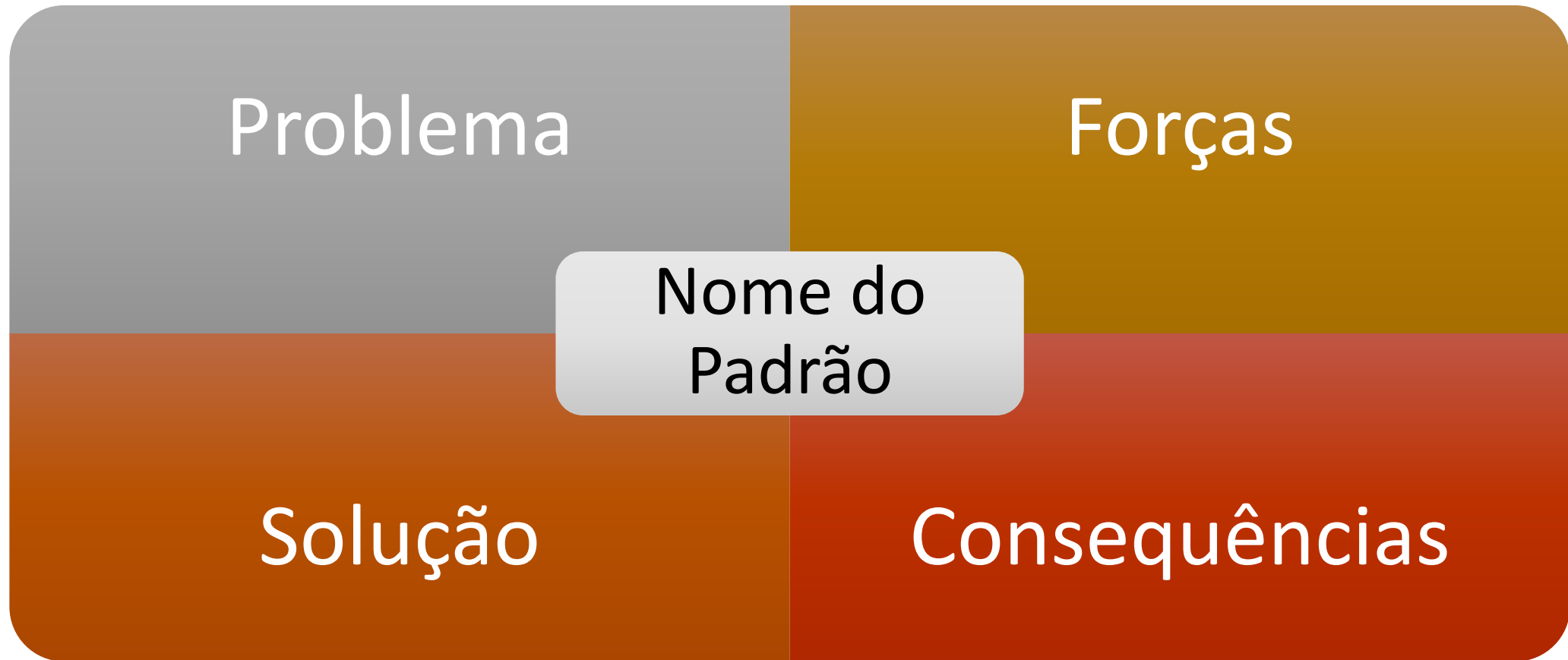
Um padrão arquitetural expressa um esquema de organização estrutural fundamental para sistemas de software. Ele fornece um conjunto de subsistemas predefinidos, especifica suas responsabilidades e inclui regras e diretrizes para organizar as relações entre eles.

Exemplo de Padrão Arquitetural

Arquitetura Limpa



Elementos essenciais de um *padrão*





Um exemplo: *Problema*

- Você chega em casa do trabalho, abre a sua geladeira e vê que várias coisas estão faltando.
 - Você precisa ir ao supermercado, que é longe da sua casa.
 - A sua memória já não é a mesma de antigamente.
 - Que **padrão** você usaria para endereçar este problema?
-

Padrão

- Nome significativo para uma solução padronizada
 - *Lista de Compras*
-

Padrão

- Problema em contexto
 - Comprar vários itens para a geladeira mesmo quando a sua memória seja curta.
-

Padrão

- Forças
 - Várias pequenas compras consomem muito tempo e combustível.
 - O trânsito está caótico.
 - Deseja-se comprar tudo de uma vez!
-

Padrão

- Solução

- Verificar quais itens faltam na geladeira.
 - Anotá-los em uma lista.
 - Ir ao supermercado.
 - Comprar todos os itens da lista.
 - Abastecer a geladeira com os itens comprados.
-

Padrão

- Consequências
 - Vantagem:
 - Maior eficácia nas compras.
 - Desvantagens:
 - A lista de compras pode ter um preço proibitivo.
 - Organizar uma lista exige planejamento e atenção.
-



Curiosidade

- Se fosse um problema de software, nosso padrão “**Lista de Compras**” se chamaria “**Batch Method**”,
- Esse padrão é muito usado para evitar múltiplas chamadas de rede a banco de dados.

Recursos sobre padrões para arquitetos de software

- Gamma, E. (2009). *Padrões de projetos: soluções reutilizáveis*. Bookman editora.
- Fowler, M. (2009). *Padrões de arquitetura de aplicações corporativas*. Bookman.
- Buschmann, F., Henney, K., & Schmidt, D. C. (2007). *Pattern-oriented software architecture, on patterns and pattern languages* (Vol. 5). John wiley & sons.
- Alexander, C. (1979). *The timeless way of building* (Vol. 1). New York: Oxford University Press.

Resumo

- Padrões são melhores práticas
- As melhores arquiteturas estão repletas de padrões
- Aplique os padrões gentilmente, considerando sempre o contexto, forças e consequências.



Desenho Dirigido por Domínios

Marco Mendes

DDD – Domain Driven Design

Padrão arquitetural
para a organização do
domínio da aplicação.

Tem por objetivo criar
sistemas com regras de
negócios mais simples
de manter e testar.

“Especialistas do domínio (usuários, analistas e outros especialistas da área), juntamente com os desenvolvedores e arquitetos trabalham de mãos dadas com um objetivo em comum: construir software guiado pelo domínio para atender as necessidades do cliente.

Para fazer isso, em primeiro lugar, é necessário que todos usem uma linguagem em comum e que não haja tradução na comunicação entre os membros do time.

O time desenvolve, portanto, uma linguagem ubíqua (geral e universal) que expressa o conhecimento dos especialistas de negócio para o modelo de domínio (para o código).”,

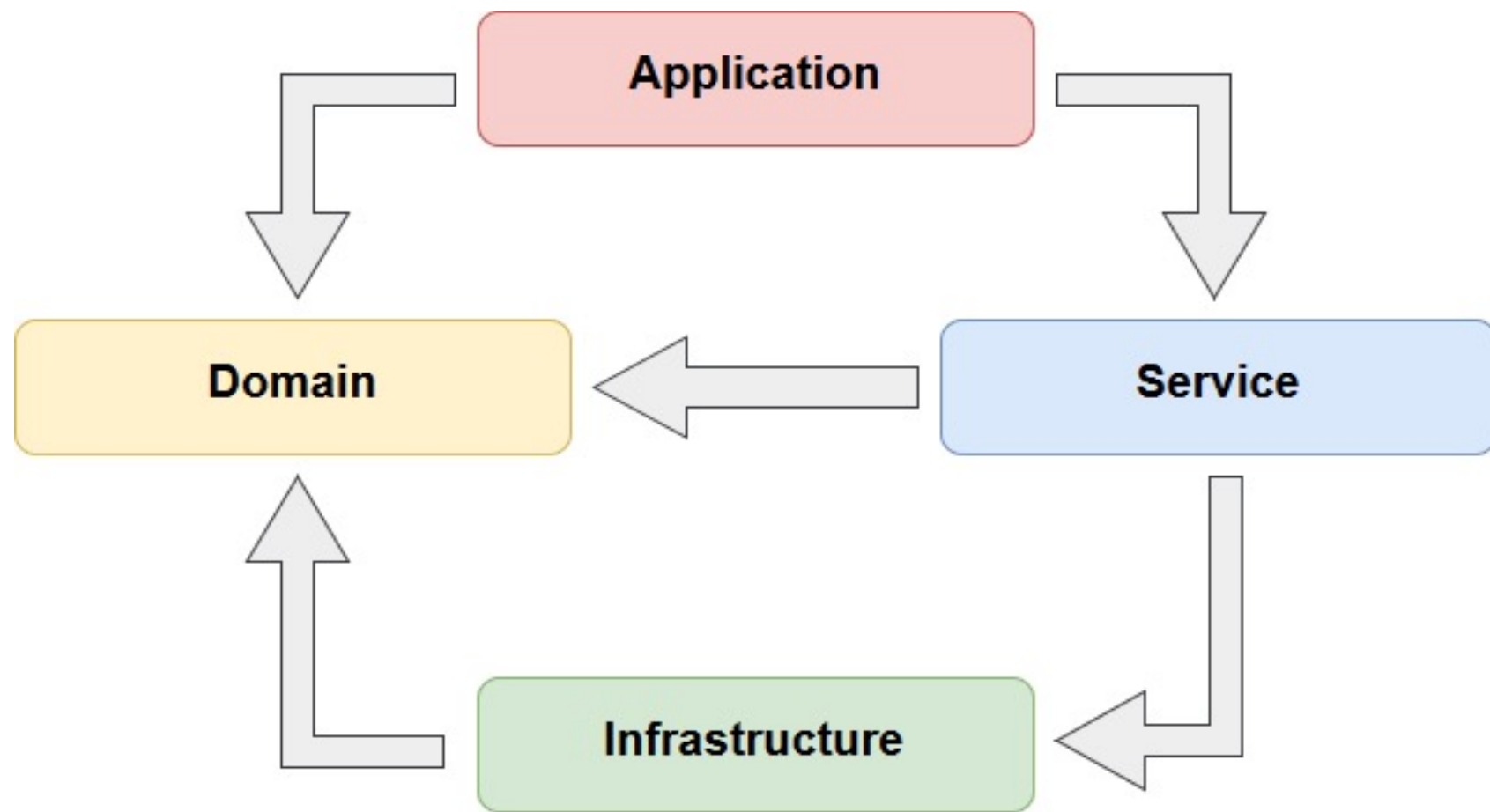
Eric Evans

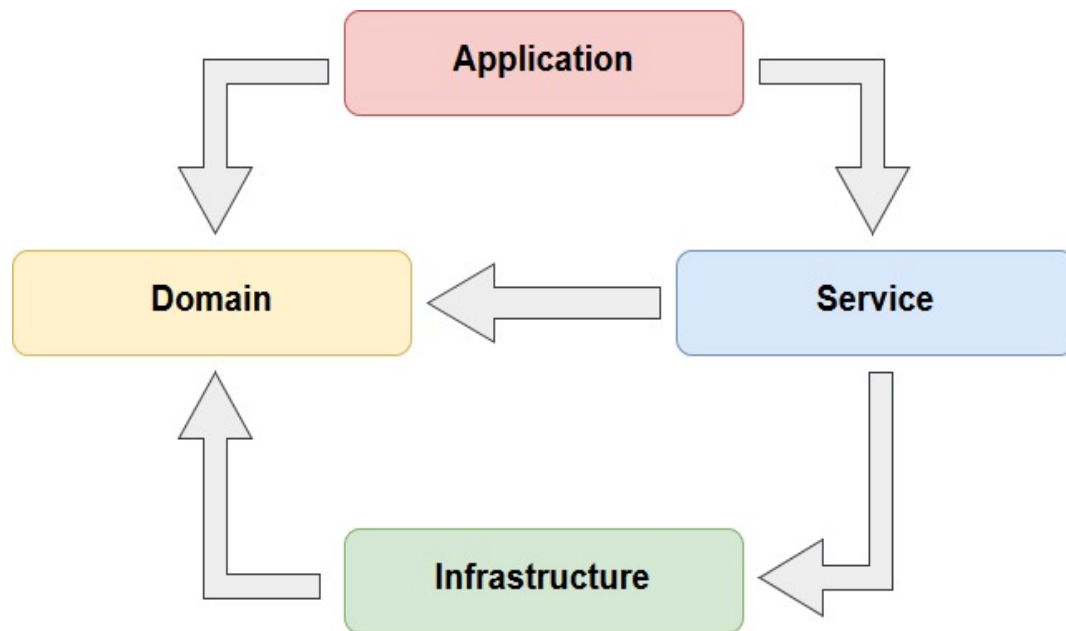
Regras para criar um modelo efetivo

Vincular o **modelo** com a **implementação**: esse vínculo é feito desde o início, quando o modelo ainda é primitivo e será mantido até o fim.

A implementação deve refletir integralmente o modelo.

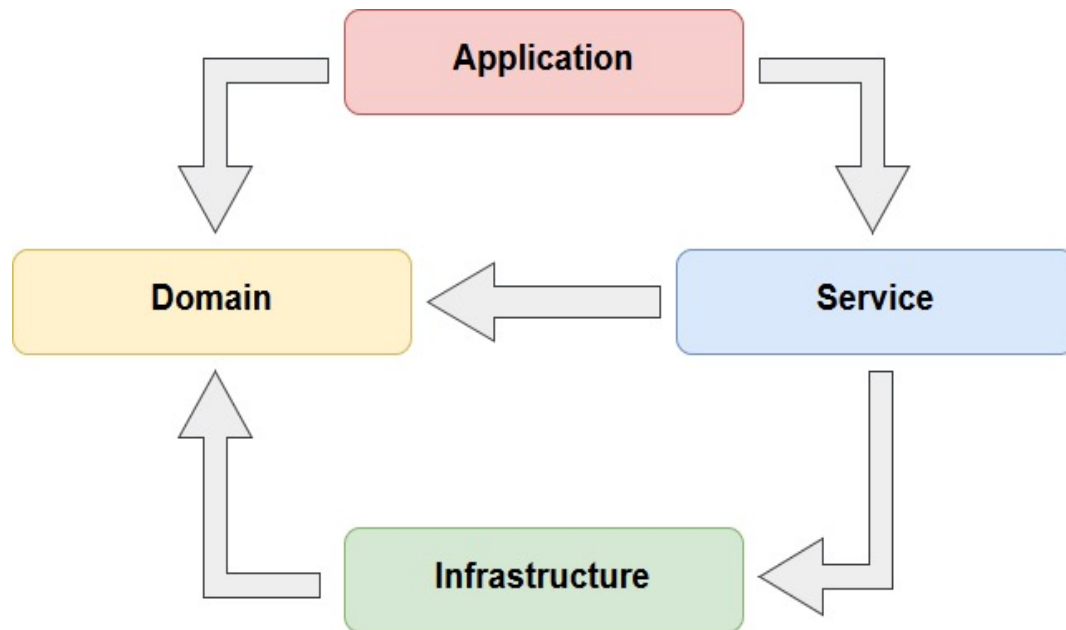
Cultivar uma linguagem baseada no modelo: no início será necessário que os desenvolvedores e os *domain experts* entendam cada um os termos do outro, mas depois ambos falarão a mesma linguagem, organizando as sentenças da comunicação numa estrutura consistente com o modelo e sem ambiguidades.





Camada de aplicação

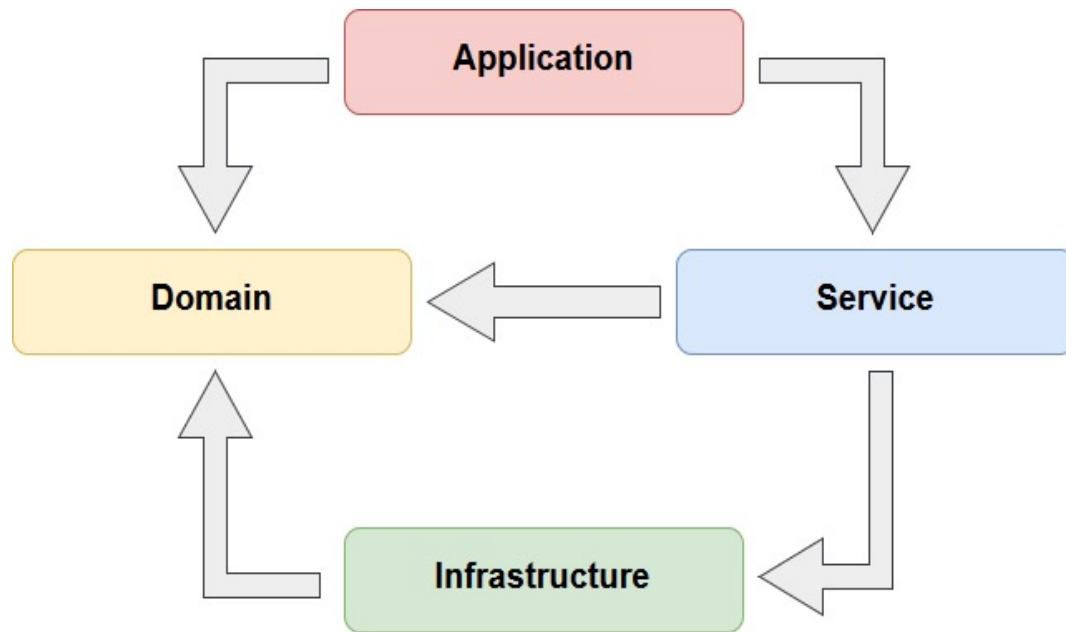
Responsável pelo projeto principal. Aqui serão implementados os controladores e a exposição de APIs. Tem a função de receber todas as requisições e direcioná-las a algum Service para executar uma determinada ação.



Camada de serviço

Nela que são implementadas fluxos de trabalho e regras globais ao sistema.

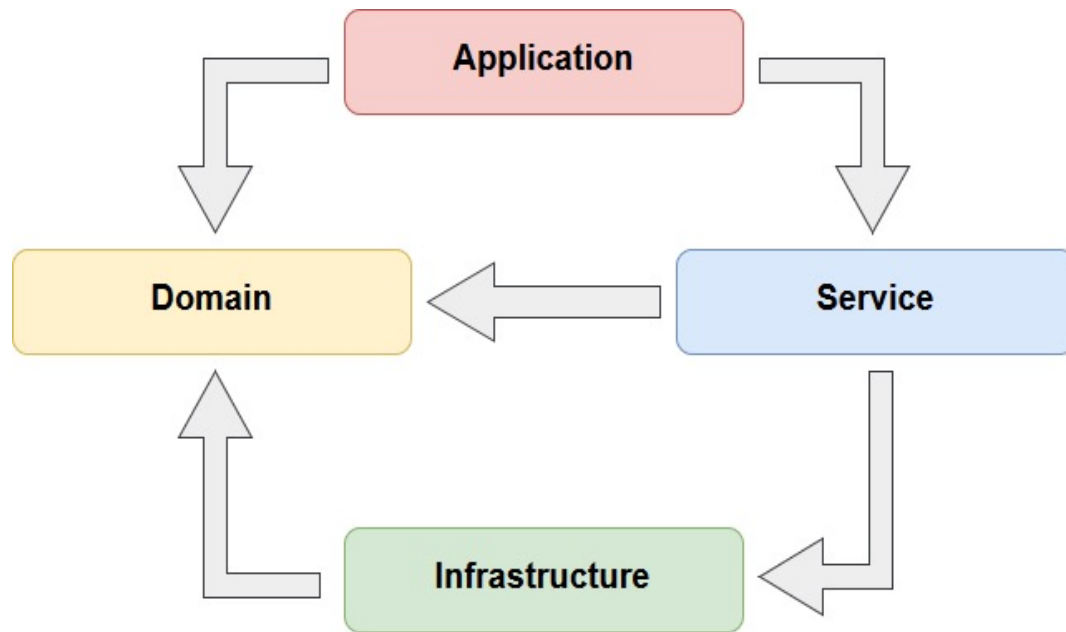
Essa camada também usa o domínio para persistir informações contra os bancos de dados.



Camada de infraestrutura

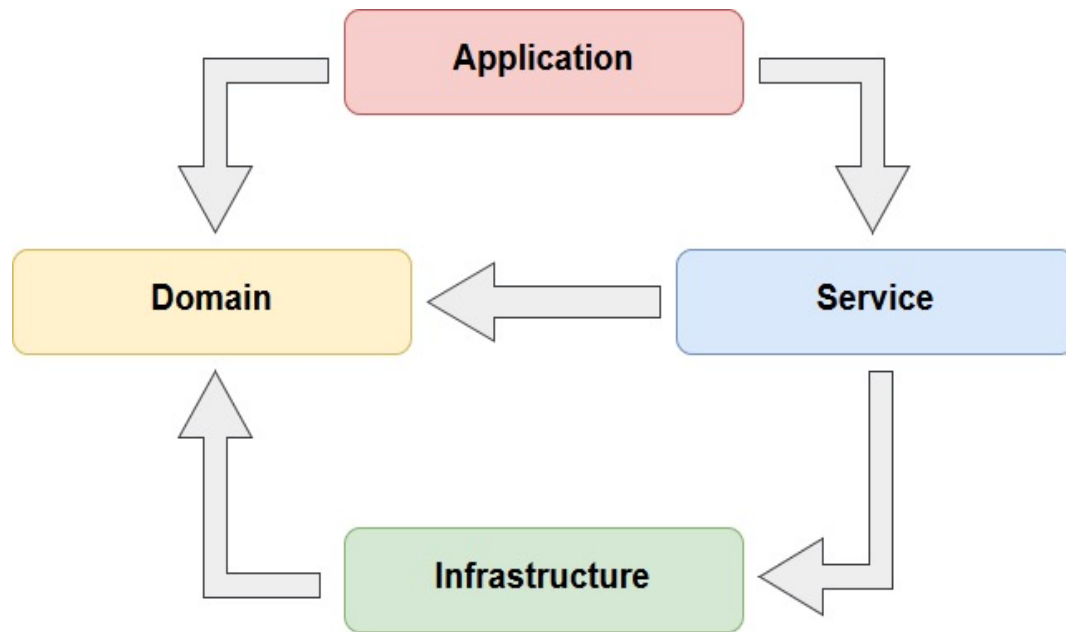
É dividida em duas subcamadas

- *Data*: realiza a persistência com o banco de dados, utilizando, ou não, algum *ORM*.
- *Cross-Cutting*: uma camada 'separada' que não obedece a hierarquia de camada. Como o próprio nome diz, essa camada cruza toda a hierarquia. Contém as funcionalidades que pode ser utilizadas em qualquer parte do código, como, por exemplo, validação de CPF/CNPJ, consumo de API externa e utilização de alguma biblioteca de segurança. *Possui referências da camada Domain*.



Camada de domínio

- Ela é responsável por representar conceitos, informações e situações referentes aos negócios.
- O estado que reflete a situação de negócios é controlado e usado aqui, embora os detalhes técnicos sobre como armazená-lo sejam delegados à infraestrutura. Essa camada é a essência de negócio do software sendo construído.
- A camada de modelo de domínio é a implementação do negócio. Quando você implementa uma camada de modelo de domínio de microsserviço em alguma linguagem, essa camada é codificada como uma biblioteca de classes reusável (DLL ou JAR), com as entidades de domínio que modelam os dados e realizam operações sobre ele.



Camada de domínio

- Seguindo os princípios de [Ignorância da Persistência](#) e a [Ignorância da Infraestrutura](#), essa camada deve ignorar totalmente os detalhes tecnológicos. Essas tarefas de persistência devem ser executadas pela camada de infraestrutura.
- Entidades de domínio não devem ter nenhuma dependência direta (como derivar de uma classe base) da infraestrutura ou acesso a dados, como Entity Framework, NHibernate ou JPA.
- Idealmente, suas entidades de domínio não devem derivar nem implementar nenhum tipo definido na infraestrutura.

Regras para criar um modelo efetivo

- ❖ **Desenvolver um modelo rico em conhecimento:** objetos têm dados e comportamentos associados. O modelo não deve ser apenas uma estrutura de dados (modelo anêmico), ele deve capturar o conhecimento do domínio para resolver os problemas do domínio.
- ❖ **Refinar o modelo:** o modelo deve ser refinado. Assim como conceitos importantes devem ser adicionados, conceitos que não tem relevância devem ser removidos. **A cada iteração o modelo ficará mais rico e terá mais valor.**
- ❖ **Brainstorming e experimentação:** a interação direta entre os desenvolvedores e domain experts, através de brainstormings e diagramas feitos na hora, transformam as discussões em **laboratórios do modelo**, no qual diversas variações de experimentos podem ser exercitadas.

Referências adicionais

- Implementação em Java
 - <http://www.agileandart.com/2010/07/16/ddd-introducao-a-domain-driven-design/>
 - <https://github.com/ddd-by-examples/library>
- Implementação em C#
 - <https://paulovich.net/clean-architecture-for-net-applications/>
 - <https://docs.microsoft.com/pt-br/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/ddd-oriented-microservice>
 - <https://docs.microsoft.com/pt-br/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/microservice-domain-model>
 - <https://docs.microsoft.com/pt-br/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/net-core-microservice-domain-model>
- Eshop On Containers
<https://github.com/dotnet-architecture/eShopOnContainers>

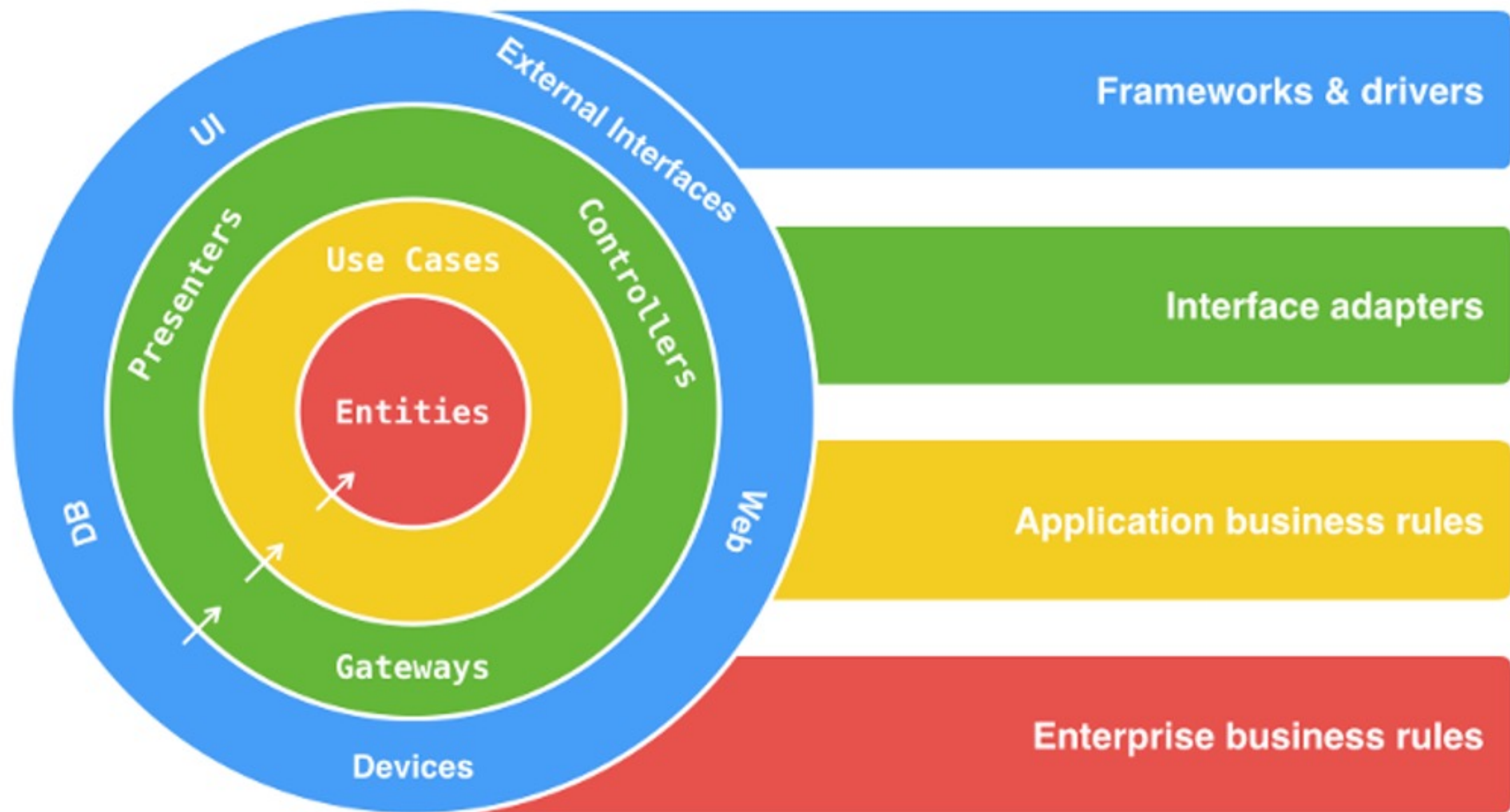
Resumo

- O DDD busca criar modelos de domínios mais resilientes à mudanças tecnológicas.
- O DDD é baseado em um conjunto de padrões arquiteturais e padrões de desenho que buscam organizar domínios, serviços, infraestrutura tecnológica e aplicações.
- Requer um time com mais experiência em OO, padrões de desenho e padrões de arquitetura.



Arquitetura Limpa

Marco Mendes

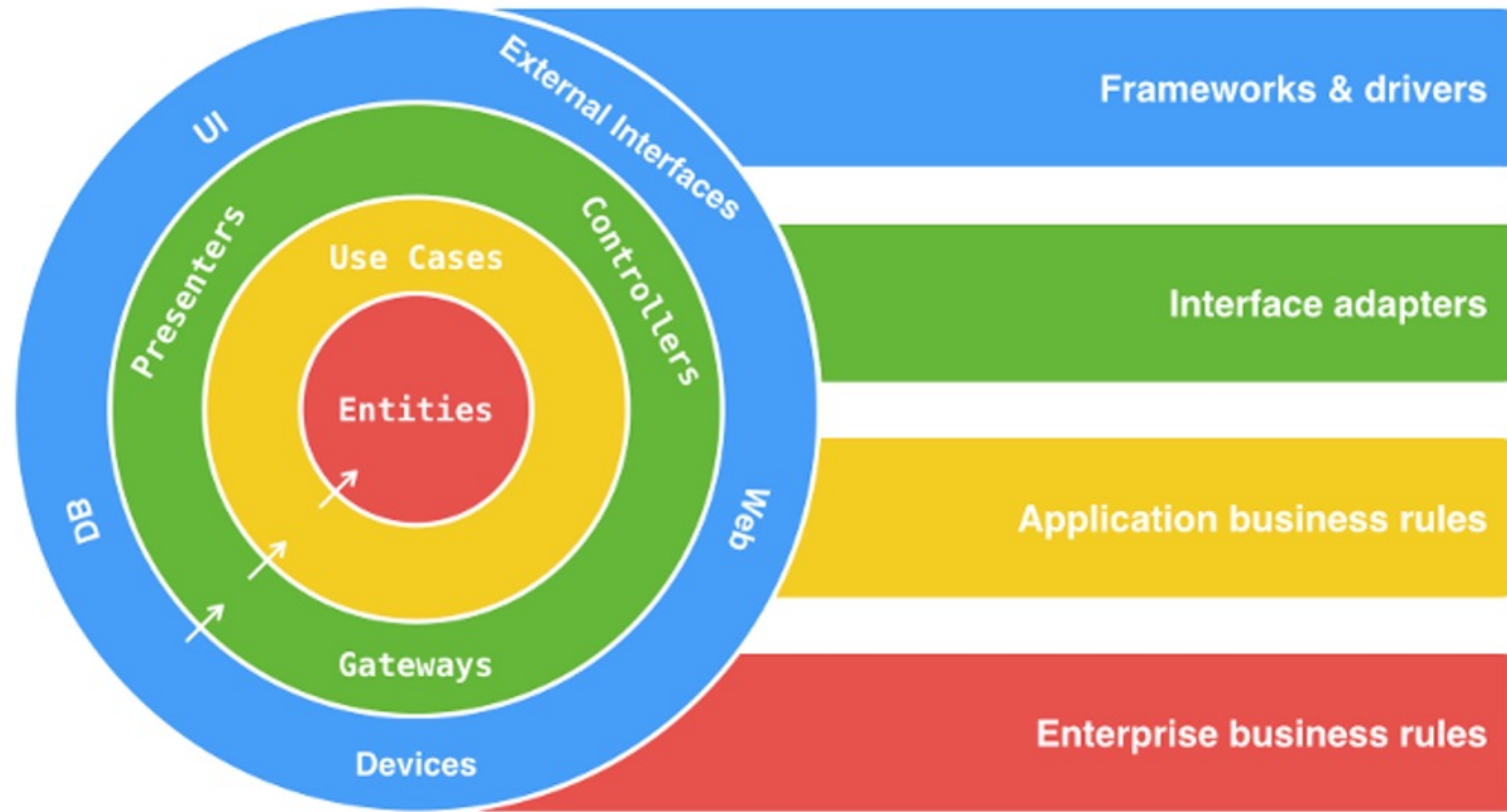


Objetivos da Arquitetura Limpa

- Independente de frameworks.
 - A arquitetura não depende da existência de alguma biblioteca de software carregado de recursos. Isso permite que você use essas estruturas como ferramentas, ao invés de ter que amontoar seu sistema em suas restrições limitadas.
- Testabilidade facilitada.
 - As regras de negócios podem ser testadas sem a IU, Banco de Dados, Servidor Web ou qualquer outro elemento externo.

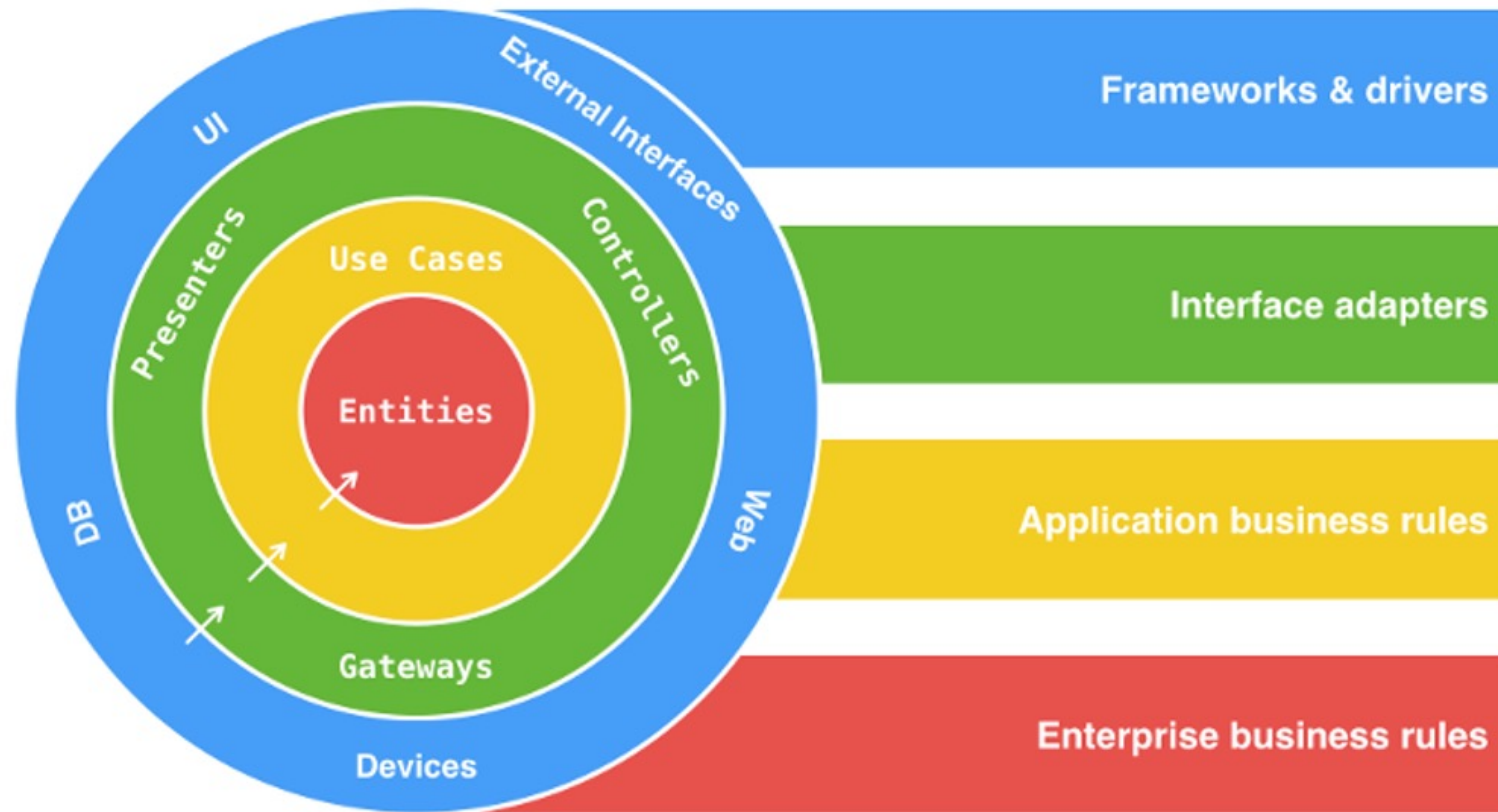
Objetivos da Arquitetura Limpa

- Independente de banco de dados.
 - Você pode trocar Oracle ou SQL Server, por Mongo, BigTable, CouchDB ou qualquer outro.
 - Suas regras de negócios não estão vinculadas ao banco de dados.
- Independente de qualquer agência externa.
 - Na verdade, suas regras de negócios simplesmente não sabem absolutamente nada sobre o mundo exterior.



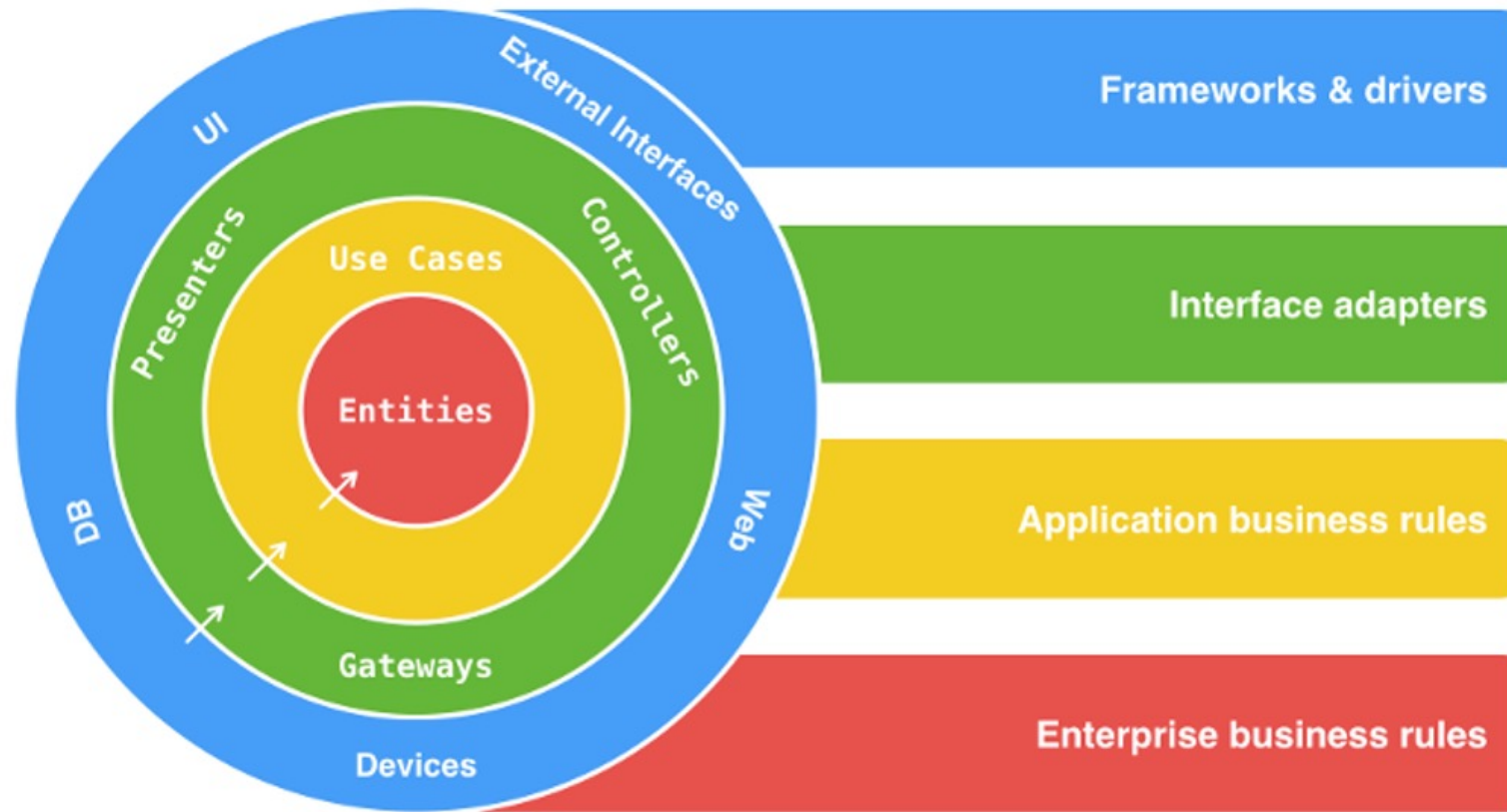
Framework e Drivers

É a camada mais externa e protege o sistema das mudanças de detalhes de implementação. Exemplos incluem ORM ou bibliotecas de segurança da informação.



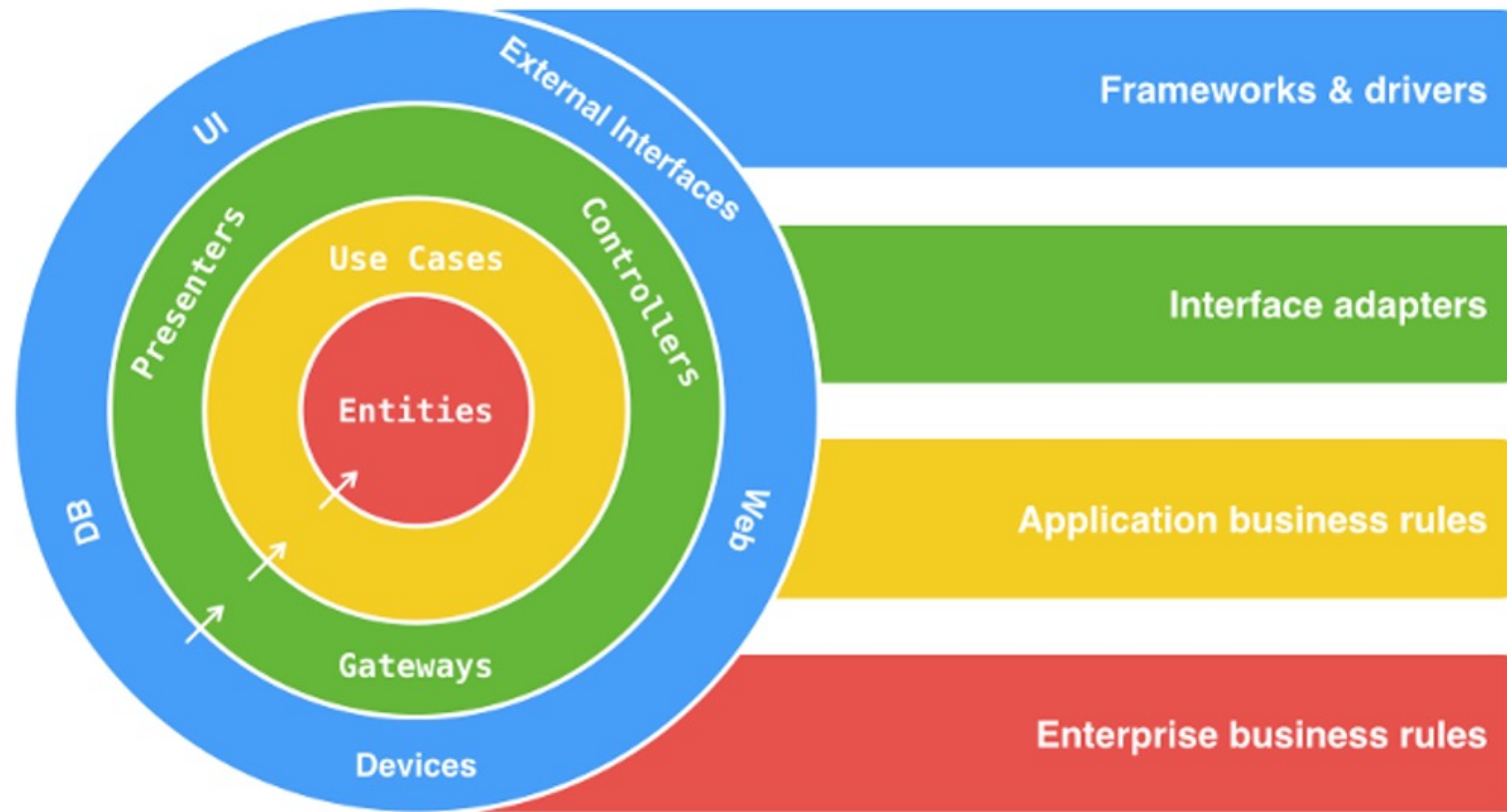
Adaptadores de Interfaces

Converte os dados informados pelos usuários em uma estrutura que interage com o sistema, é a camada onde reside os presenters, controles, gateways e repositórios.



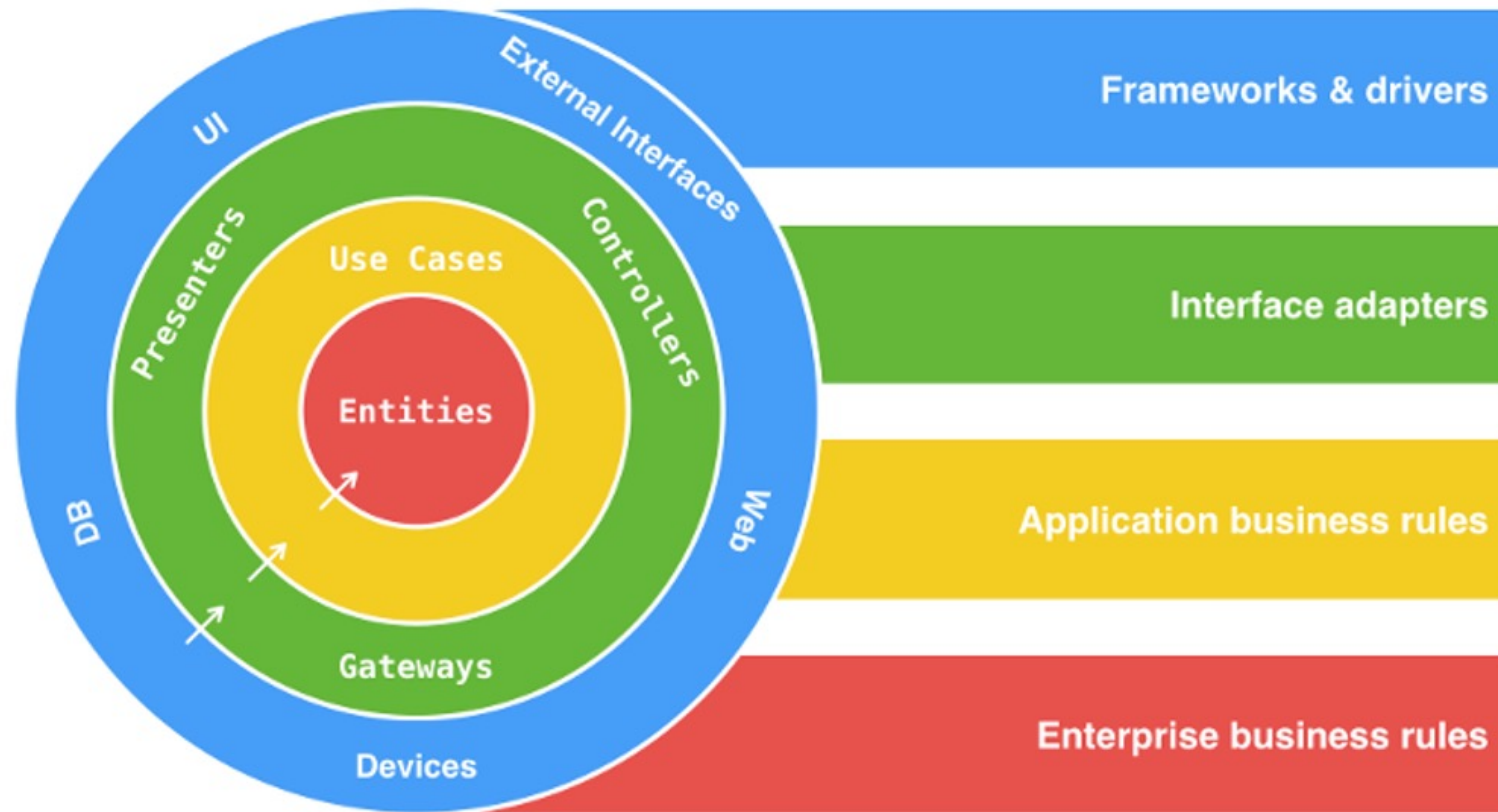
Regras de Negócio de Aplicação

É responsável pelo processamento das informações, é onde ficam os casos de uso, onde acontece a validação das regras de negócios e feedback do que está acontecendo.



Na proposta da Arquitetura limpa todas as dependências devem apontar para dentro, os círculos externos são mecanismos e os internos políticas.

Em outras palavras o código fonte não deve ter conhecimento das camadas externas a sua camada, não devem ter nenhum tipo de dependência a estas por isso o nome de Regra da Dependência.



Regras de Negócio Corporativas

Camada das *entidades* onde ficam os códigos de regra de negócio de nível de negócio. Esses códigos devem estar prontos para o reaproveitamento e compõe a parte mais nobre do sistema.

Referências adicionais

- Arquiteturas Limpas
 - <https://dzone.com/articles/four-architectures-will>
 - <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
 - Exemplo em Node - <https://github.com/howardmann/clean-node>
 - Exemplo em .NET - <https://github.com/ivanpaulovich/clean-architecture-manga>
 - Exemplo em Java - <https://www.baeldung.com/spring-boot-clean-architecture>
- Arquitetura Hexagonal (Alistair Cockburn)
 - <https://alistair.cockburn.us/hexagonal-architecture/>
 - <https://herbertograca.com/2017/11/16/explicit-architecture-01-ddd-hexagonal-onion-clean-cqrs-how-i-put-it-all-together/>

Resumo

- Objetivos
 - Regras de negócio sem acoplamento com banco de dados e outros detalhes de tecnologia.
 - Tornar os testes de negócio mais simples.
- Requer um time com mais experiência em OO, padrões de desenho e padrões de arquitetura.



Padrões de Desenho no DDD

Marco Mendes

Padrões essenciais do DDD

Objeto de
Valor

Entidade

Raiz
agregada

Repositório

Caso de
Uso

Contexto
Limitado

Fábrica de
Entidades

Serviço de
Domínio

Objeto de Valor (VO/Value Object)

- Encapsula as minúsculas regras de negócios do domínio.
- Estruturas que são únicas por suas propriedades e todo o objeto é imutável, uma vez criado, seu estado não pode mudar.

Objeto de Valor/VO

```
public readonly struct Name
{
    private readonly string _text;

    public Name(string text)
    {
        if (string.IsNullOrEmpty(text))
            throw new NameShouldNotBeEmptyException("The 'Name' field is required");

        _text = text;
    }

    public override string ToString()
    {
        return _text;
    }
}
```

Entidade (Entities)

- Encapsula as regras de negócios do domínio.
- Objetos altamente abstratos e mutáveis identificados por seus chaves primárias.

Entidades

```
public abstract class Credit : ICredit
{
    /// <summary>
    /// Gets or sets Id.
    /// </summary>
    public CreditId Id { get; protected set; }

    /// <summary>
    /// Gets or sets Amount.
    /// </summary>
    public PositiveMoney Amount { get; protected set; }

    /// <summary>
    /// Gets Description.
    /// </summary>
    public string Description
    {
        get { return "Credit"; }
    }

    /// <summary>
    /// Gets or sets Transaction Date.
    /// </summary>
    public DateTime TransactionDate { get; protected set; }

    /// <summary>
    /// Calculate the sum of positive amounts.
    /// </summary>
    /// <param name="amount">Positive amount.</param>
    /// <returns>The positive sum.</returns>
    public PositiveMoney Sum(PositiveMoney amount)
    {
        return this.Amount.Add(amount);
    }
}
```


Raiz Agregada (*Agregate Root*)

- Semelhante às Entidades com a adição de que a Raiz Agregada é responsável por manter o grafo de objetos que ela contém consistente.

Raiz Agregada

```
public abstract class Account : IAccount
{
    /// <summary>
    /// Initializes a new instance of the <see cref="Account"/> class.
    /// </summary>
    protected Account()
    {
        this.Credits = new CreditsCollection();
        this.Debits = new DebitsCollection();
    }

    /// <inheritdoc/>
    public AccountId Id { get; protected set; }

    /// <summary>
    /// Gets or sets Credits List.
    /// </summary>
    public CreditsCollection Credits { get; protected set; }

    /// <summary>
    /// Gets or sets Debits List.
    /// </summary>
    public DebitsCollection Debits { get; protected set; }

    /// <inheritdoc/>
    public ICredit Deposit(IAccountFactory entityFactory, PositiveMoney amountToDeposit)
    {
        var credit = entityFactory.NewCredit(this, amountToDeposit, DateTime.UtcNow);
        this.Credits.Add(credit);
        return credit;
    }
}
```

Raiz Agregada

```
/// <inheritdoc/>
public IDebit Withdraw(IAccountFactory entityFactory, PositiveMoney amountToWithdraw)
{
    if (this.GetCurrentBalance().LessThan(amountToWithdraw))
    {
        throw new MoneyShouldBePositiveException("Account has not enough funds.");
    }

    var debit = entityFactory.NewDebit(this, amountToWithdraw, DateTime.UtcNow);
    this.Debits.Add(debit);
    return debit;
}

/// <inheritdoc/>
public bool IsClosingAllowed()
{
    return this.GetCurrentBalance().IsZero();
}

/// <inheritdoc/>
public Money GetCurrentBalance()
{
    var totalCredits = this.Credits
        .GetTotal();

    var totalDebits = this.Debits
        .GetTotal();

    var totalAmount = totalCredits
        .Subtract(totalDebits);

    return totalAmount;
}
}
```

Repositorios (*Repositories*)

- Fornece recursos de persistência para as Raízes Agregadas.

Repositórios

```
public sealed class CustomerRepository : ICustomerRepository
{
    private readonly MangaContext _context;

    public CustomerRepository(MangaContext context)
    {
        _context = context;
    }

    public async Task Add(ICustomer customer)
    {
        _context.Customers.Add((InMemoryDataAccess.Customer) customer);
        await Task.CompletedTask;
    }

    public async Task<ICustomer> Get(Guid id)
    {
        Customer customer = _context.Customers
            .Where(e => e.Id == id)
            .SingleOrDefault();

        return await Task.FromResult<Customer>(customer);
    }

    public async Task Update(ICustomer customer)
    {
        Customer customerOld = _context.Customers
            .Where(e => e.Id == customer.Id)
            .SingleOrDefault();

        customerOld = (Customer) customer;
        await Task.CompletedTask;
    }
}
```

Serviço de Domínio (*Domain Service*)

- Fornece para a camada de aplicação métodos que permitam a execução de operações sobre os objetos de Domínio (camada mais interna).

Serviço de Domínio

```
public class AccountService
{
    private readonly IAccountFactory _accountFactory;
    private readonly IAccountRepository _accountRepository;

    /// <summary>
    /// Initializes a new instance of the <see cref="AccountService"/> class.
    /// </summary>
    /// <param name="accountFactory">Account Factory.</param>
    /// <param name="accountRepository">Account Repository.</param>
    public AccountService(
        IAccountFactory accountFactory,
        IAccountRepository accountRepository)
    {
        this._accountFactory = accountFactory;
        this._accountRepository = accountRepository;
    }

    /// <summary>
    /// Open Checking Account.
    /// </summary>
    /// <param name="customerId">Customer Id.</param>
    /// <param name="amount">Amount.</param>
    /// <returns>IAccount created.</returns>
    public async Task<IAccount> OpenCheckingAccount(CustomerId customerId, PositiveMoney amount)
    {
        var account = this._accountFactory.NewAccount(customerId);
        var credit = account.Deposit(this._accountFactory, amount);
        await this._accountRepository.Add(account, credit);

        return account;
    }
}
```


Serviço de Domínio

```
/// <summary>
/// Withdrawls from Account.
/// </summary>
/// <param name="account">Account.</param>
/// <param name="amount">Amount.</param>
/// <returns>Debit Transaction.</returns>
public async Task<IDebit> Withdraw(IAccount account, PositiveMoney amount)
{
    var debit = account.Withdraw(this._accountFactory, amount);
    await this._accountRepository.Update(account, debit);

    return debit;
}

/// <summary>
/// Deposits into Account.
/// </summary>
/// <param name="account">Account.</param>
/// <param name="amount">Amount.</param>
/// <returns>Credit Transaction.</returns>
public async Task<ICredit> Deposit(IAccount account, PositiveMoney amount)
{
    var credit = account.Deposit(this._accountFactory, amount);
    await this._accountRepository.Update(account, credit);

    return credit;
}
}
```


Casos de Uso

(Use Cases)

- É o ponto de entrada do aplicativo para uma interação do usuário.
- Ele aceita uma mensagem de entrada, executa o algoritmo e deve fornecer a mensagem de saída para a porta de saída.

Casos de Uso

```
public sealed class Withdraw : IUseCase
{
    private readonly AccountService _accountService;
    private readonly IOutputPort _outputPort;
    private readonly IAccountRepository _accountRepository;
    private readonly IUnitOfWork _unitOfWork;

    /// <summary>
    /// Initializes a new instance of the <see cref="Withdraw"/> class.
    /// </summary>
    /// <param name="accountService">Account Service.</param>
    /// <param name="outputPort">Output Port.</param>
    /// <param name="accountRepository">Account Repository.</param>
    /// <param name="unitOfWork">Unit Of Work.</param>
    public Withdraw(
        AccountService accountService,
        IOutputPort outputPort,
        IAccountRepository accountRepository,
        IUnitOfWork unitOfWork)
    {
        this._accountService = accountService;
        this._outputPort = outputPort;
        this._accountRepository = accountRepository;
        this._unitOfWork = unitOfWork;
    }
}
```

Casos de Uso

```
/// <summary>
/// Executes the Use Case.
/// </summary>
/// <param name="input">Input Message.</param>
/// <returns>Task.</returns>
public async Task Execute(WithdrawInput input)
{
    try
    {
        var account = await this._accountRepository.Get(input.AccountId);
        var debit = await this._accountService.Withdraw(account, input.Amount);

        await this._unitOfWork.Save();

        this.BuildOutput(debit, account);
    }
    catch (AccountNotFoundException notFoundEx)
    {
        this._outputPort.NotFound(notFoundEx.Message);
        return;
    }
    catch (MoneyShouldBePositiveException outOfBalanceEx)
    {
        this._outputPort.OutOfBalance(outOfBalanceEx.Message);
        return;
    }
}

private void BuildOutput(IDebit debit, IAccount account)
{
    var output = new WithdrawOutput(
        debit,
        account.GetCurrentBalance());

    this._outputPort.Standard(output);
}
}
```

Contexto Limitado (*Bounded Context*)

- É um limite lógico, semelhante a um módulo em um sistema.
- Em uma arquitetura de microsserviços, cada microsserviço especifica um contexto limitado para os objetos ali mantidos.

Fábrica de Entidades (*Entity Factory*)

- Cria novas instâncias de Entidades e Raízes Agregadas.
- Deve ser implementado pela camada de Infraestrutura.

Fábrica de Entidades

```
public sealed class EntityFactory : IUserFactory, ICustomerFactory, IAccountFactory
{
    public IAccount NewAccount(CustomerId customerId) => new Account(customerId);

    public ICredit NewCredit(
        IAccount account,
        PositiveMoney amountToDeposit,
        DateTime transactionDate) => new Credit(account, amountToDeposit, transactionDate);

    public ICustomer NewCustomer(
        SSN ssn,
        Name name) => new Customer(ssn, name);

    public IDebit NewDebit(
        IAccount account,
        PositiveMoney amountToWithdraw,
        DateTime transactionDate) => new Debit(account, amountToWithdraw, transactionDate);

    public IUser NewUser(CustomerId customerId, ExternalUserId externalUserId) => new User(customerId, ex
}
```


Fábrica de Entidades

```
public sealed class EntityFactory : IUserFactory, ICustomerFactory, IAccountFactory
{
    public IAccount NewAccount(CustomerId customerId) => new Account(customerId);

    public ICredit NewCredit(
        IAccount account,
        PositiveMoney amountToDeposit,
        DateTime transactionDate) => new Credit(account, amountToDeposit, transactionDate);

    public ICustomer NewCustomer(
        SSN ssn,
        Name name) => new Customer(ssn, name);

    public IDebit NewDebit(
        IAccount account,
        PositiveMoney amountToWithdraw,
        DateTime transactionDate) => new Debit(account, amountToWithdraw, transactionDate);

    public IUser NewUser(CustomerId customerId, ExternalUserId externalUserId) => new User(customerId, ex
}
```

Serviço de Aplicação (*Application Service*)

- Fornece para o usuário (através das interfaces) operações que o seu software pode executar, e controla a execução dessas operações através de chamadas a métodos de objetos das outras camadas.

Resumo

- DDD é composto por um conjunto de padrões que auxiliam os desenvolvedores e testadores a criarem domínios robustos.
- Exemplo completo dos fragmentos de código mostrado está nesse repositório público

<https://github.com/ivanpaulovich/clean-architecture-manga>



Padrões de Testes

Marco Mendes



Testabilidade

- Testar códigos em um sistema distribuído é mais desafiante.
- Compreender os diferentes tipos de testes e como usá-los é essencial para criar uma arquitetura eficiente com boa manutenibilidade e testabilidade.

Melhor prática – Pirâmide de Testes



- ❖ Em geral, quanto maior o grão de um teste, mais frágil e demorado será executado e mais difícil será mantido.
- ❖ Essa despesa adicional decorre do fato de que tais testes envolvem naturalmente mais partes móveis do que testes mais focados.

Conceito – Pirâmide de Testes



- ❖ O conceito da pirâmide de teste é uma maneira simples de pensar sobre o número relativo de testes que devem ser gravados em cada granularidade. Subindo pelas camadas da pirâmide, o escopo dos testes aumenta e o número de testes que devem ser escritos diminui.
- ❖ E no topo da pirâmide estão os testes exploratórios, explorando manualmente o sistema de maneiras que não foram consideradas como parte dos testes com script. O teste exploratório permite que a equipe aprenda sobre o sistema e eduque e melhore seus testes automatizados.

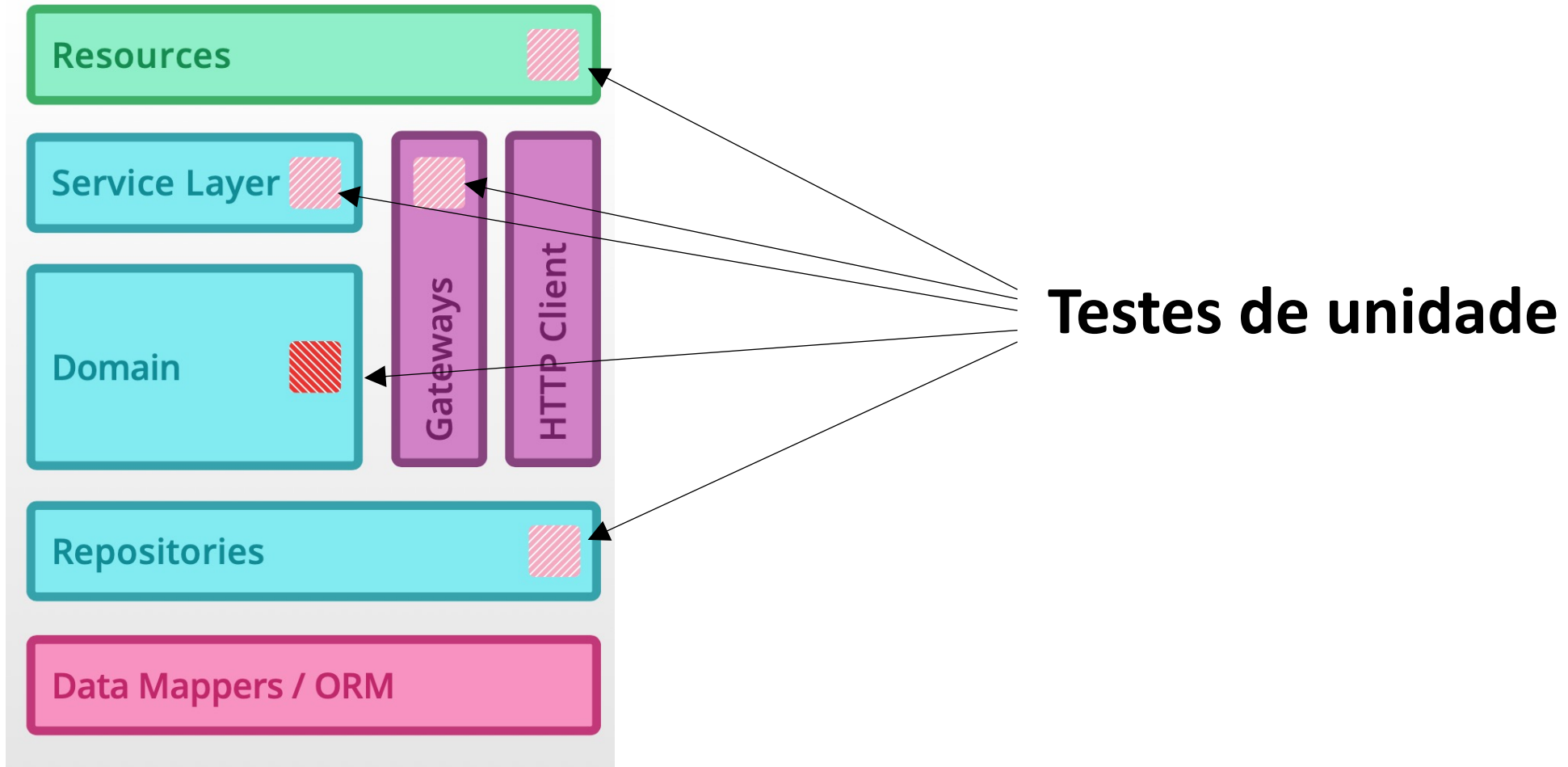


Testes de Unidade

- **Um teste de unidade exerce a menor parte do software testável no aplicativo para determinar se ele se comporta conforme o esperado.**
- O tamanho da unidade em teste não é estritamente definido; no entanto, os testes de unidade geralmente são gravados no nível de classe ou em torno de um pequeno grupo de classes relacionadas.
- Quanto menor a unidade em teste, mais fácil é expressar o comportamento usando um teste de unidade, pois a complexidade da unidade é menor.
- Muitas vezes, a dificuldade em escrever um teste de unidade pode destacar quando um módulo deve ser dividido em partes independentes mais coerentes e testadas individualmente. Assim, além de ser uma estratégia de teste útil, o teste de unidade também é uma ferramenta de design poderosa, especialmente quando combinada com abordagens TDD.



Testes de unidade



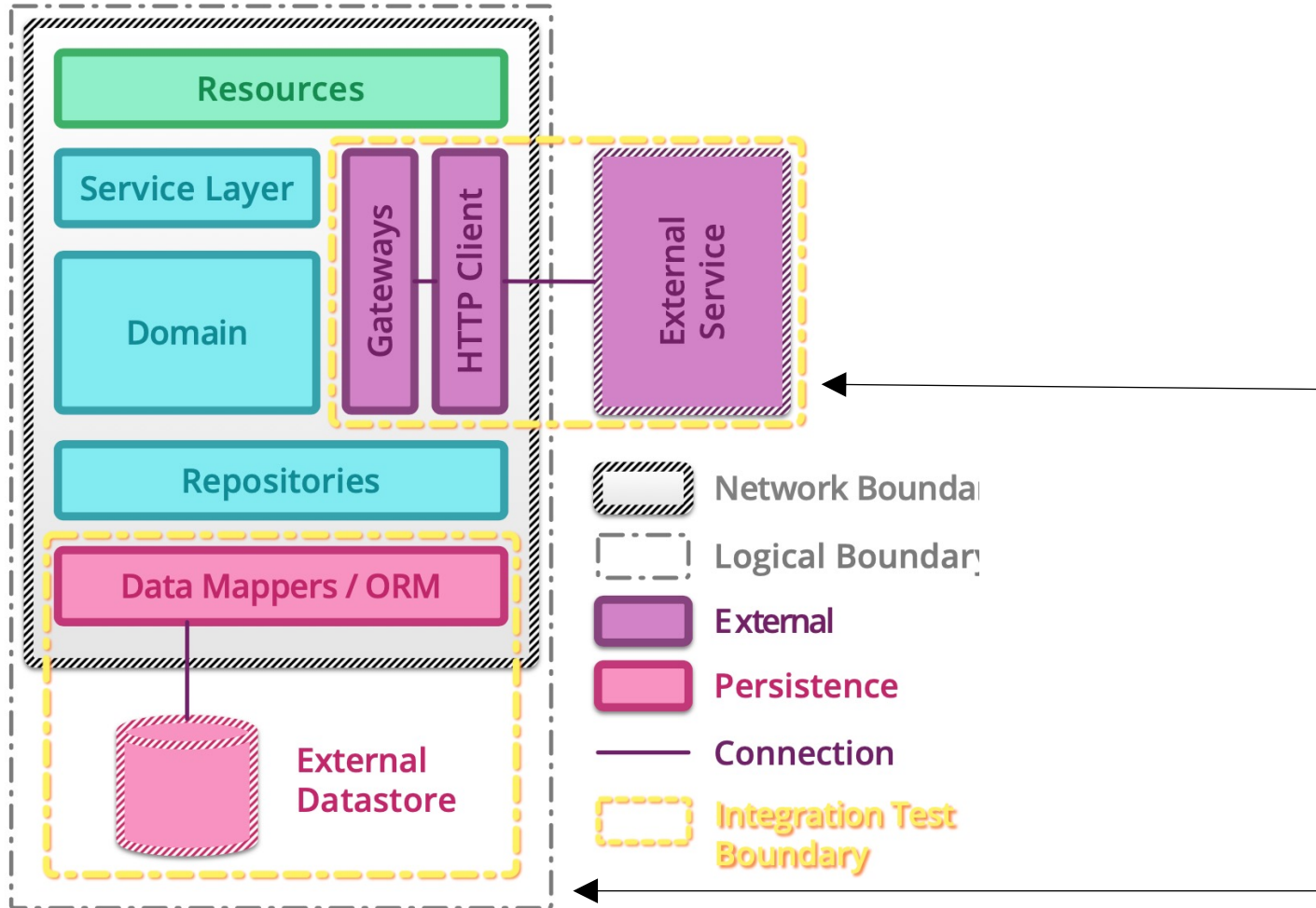
Fonte: <https://martinfowler.com/articles/microservice-testing>

Testes de Integração

- Um teste de integração verifica os caminhos de comunicação e as interações entre os componentes para detectar defeitos na interface.
- Os testes de integração coletam os módulos juntos e os testam como um subsistema, a fim de verificar se eles colaboram com o objetivo de alcançar algum tipo de comportamento maior. (exemplo: banco de dados ou caches)
- Eles exercitam caminhos de comunicação através do subsistema para verificar quaisquer suposições incorretas que cada módulo tenha sobre como interagir com seus pares.
- Isso está em contraste com testes de unidade em que, mesmo se estiver usando colaboradores reais, o objetivo é testar de perto o comportamento da unidade em teste, não o subsistema inteiro.



Testes de integração

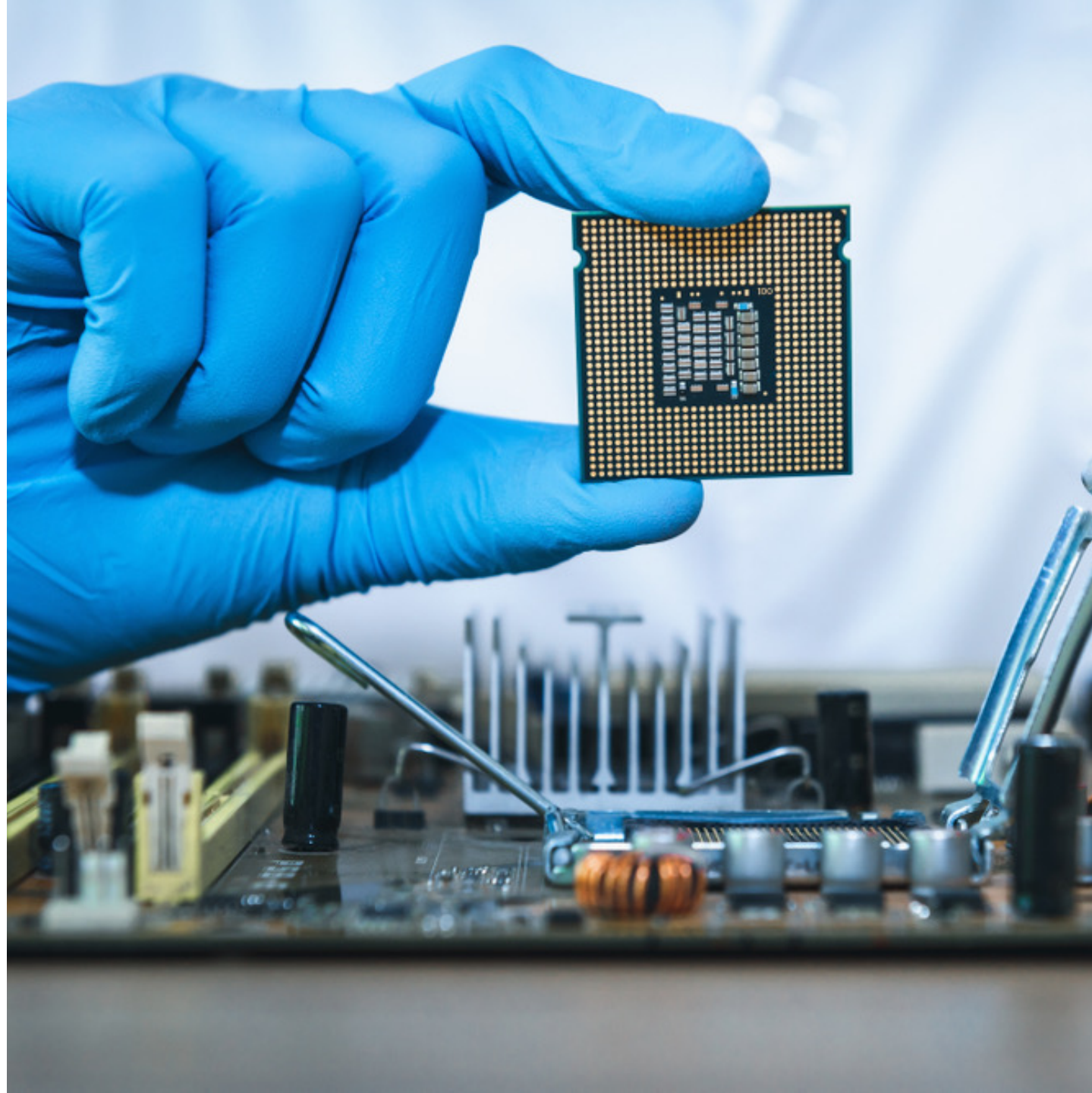


Testes de integração

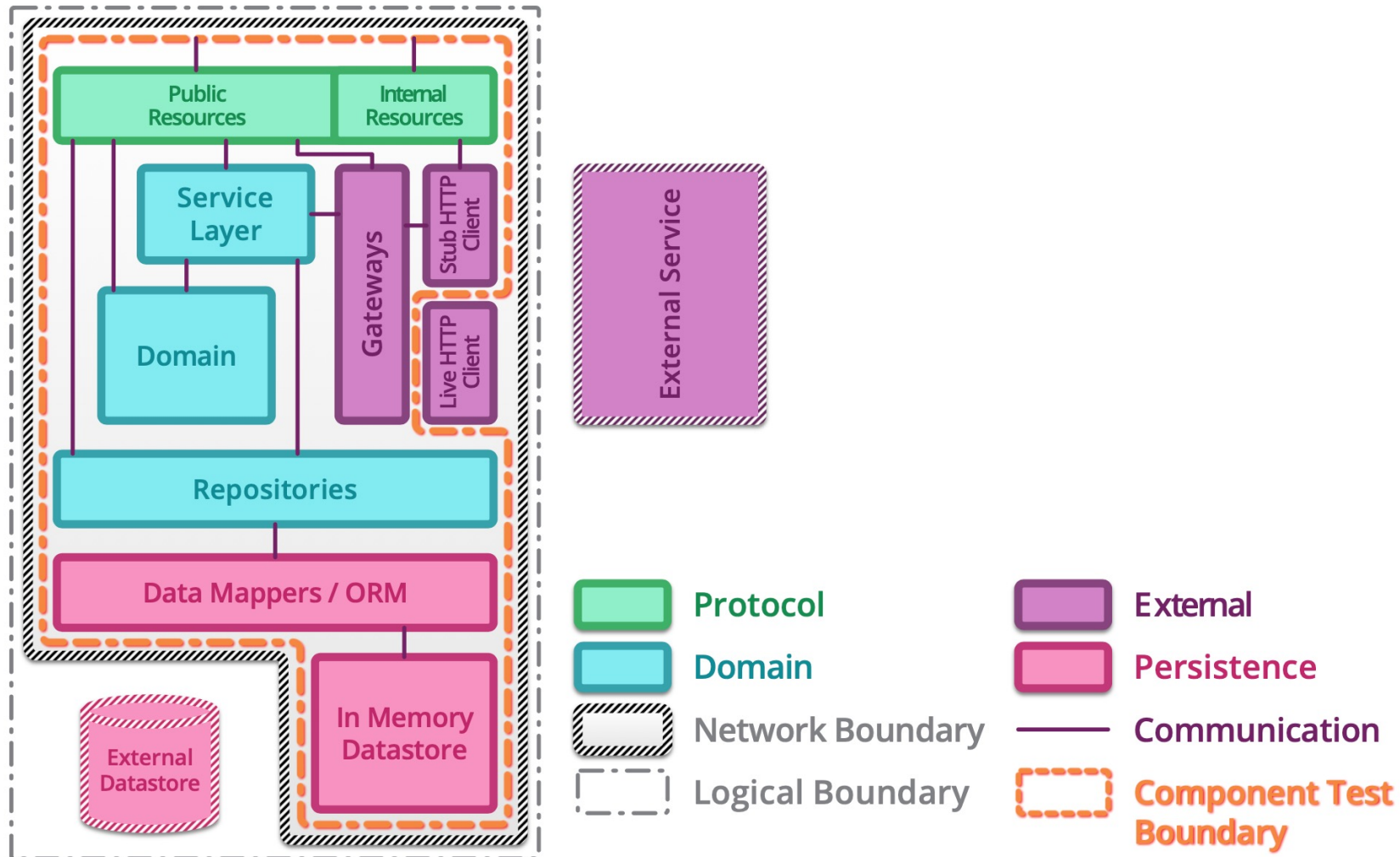
Fonte: <https://martinfowler.com/articles/microservice-testing>

Testes de componente (ou microsserviço)

- Um teste de componente limita o escopo do software exercitado a uma parte do sistema em teste, manipulando o sistema por meio de interfaces de código internas e usando dublês de teste (ex. mocks, stubs ou dummies) para isolar o código em teste de outros componentes.
- Um componente é qualquer parte bem encapsulada, coerente e independentemente substituível de um sistema maior.
- Testar esses componentes isoladamente fornece vários benefícios. Ao limitar o escopo a um único componente, é possível testar completamente o comportamento encapsulado por esse componente, mantendo ao mesmo tempo os testes que são executados mais rapidamente do que os equivalentes de pilha larga.
- Isolar o componente de seus pares usando dublês de teste evita qualquer comportamento complexo que eles possam ter. Também ajuda a fornecer um ambiente de teste controlado para o componente, acionando qualquer caso de erro aplicável de maneira repetível.

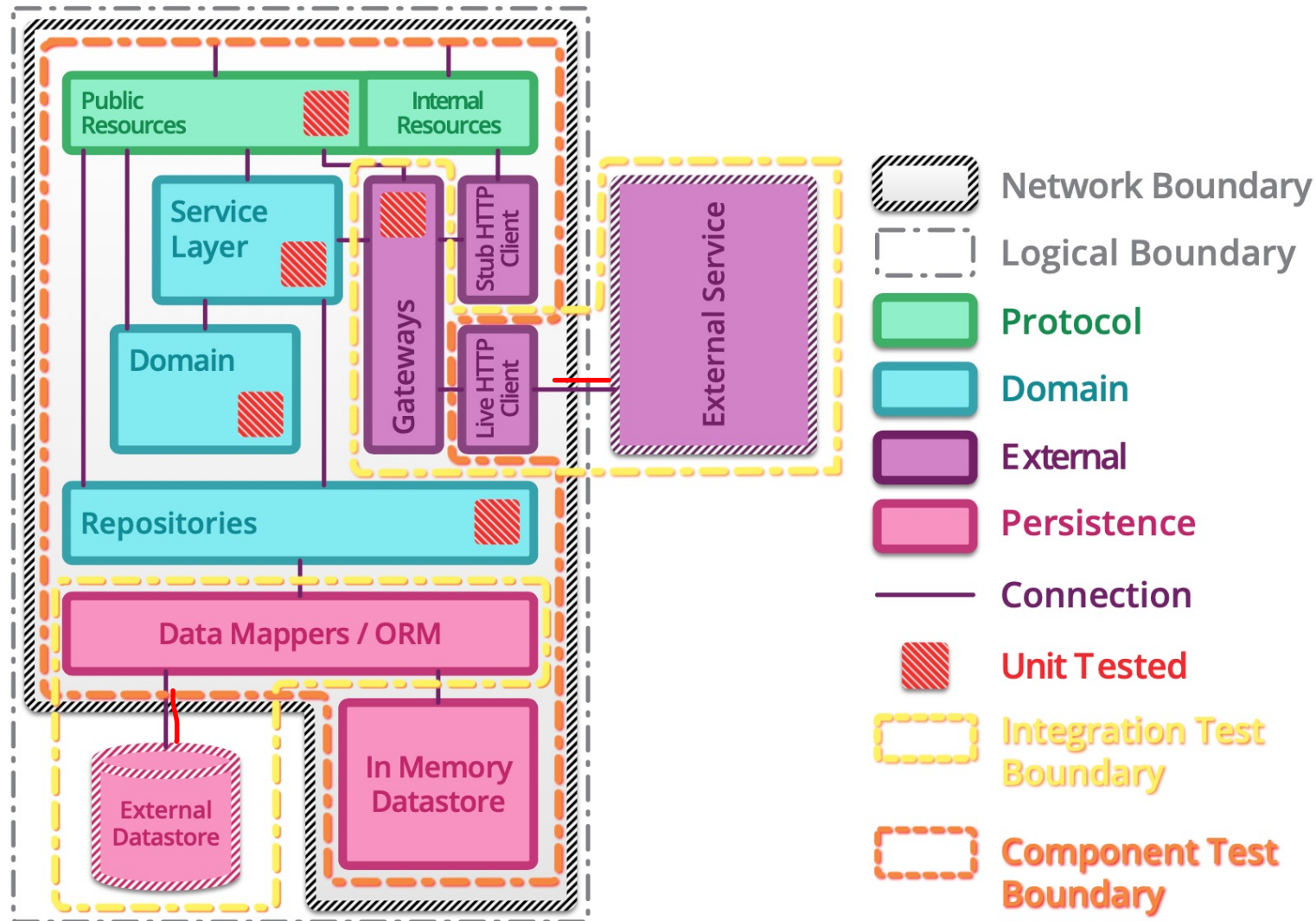


Testes de componente



Fonte: <https://martinfowler.com/articles/microservice-testing>

Combinação de testes de unidade, integração e componente fornece cobertura ampla



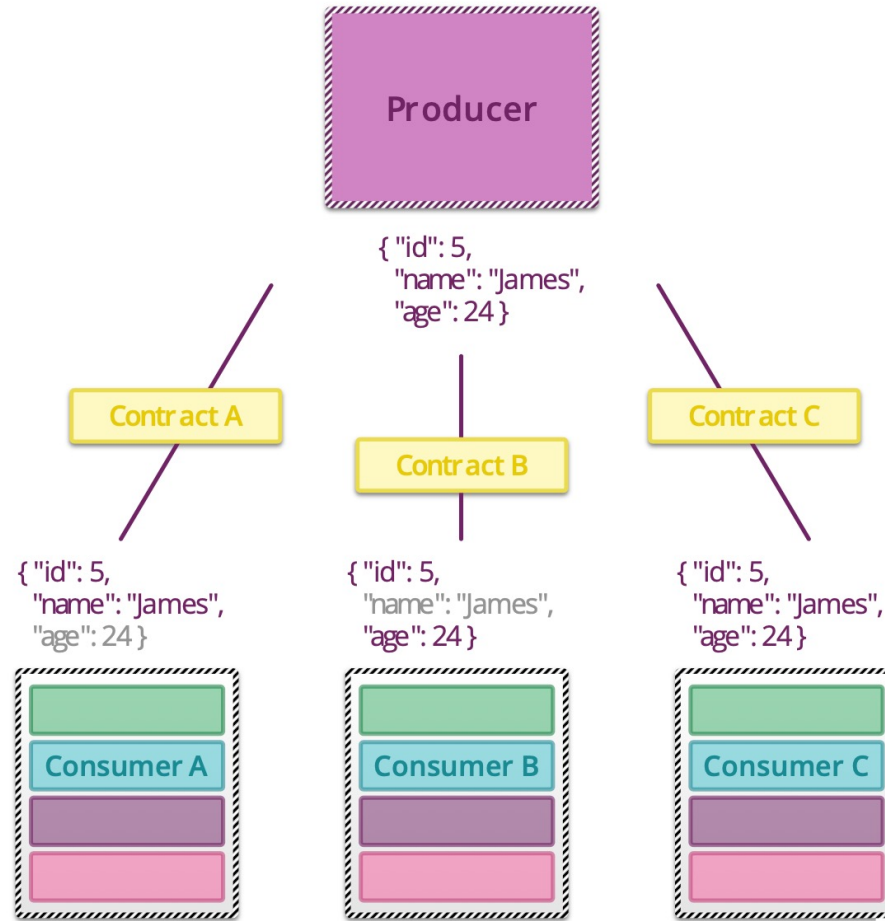
Fonte: <https://martinfowler.com/articles/microservice-testing>

Testes de contrato

- **Um teste de contrato de integração é um teste no limite de um serviço externo, verificando se ele atende ao contrato esperado por um serviço consumidor.**
- Sempre que algum consumidor se acopla à interface de um componente para fazer uso de seu comportamento, um contrato é formado entre eles. Este contrato consiste em expectativas de estruturas de dados de entrada e saída, efeitos colaterais e características de desempenho e simultaneidade.
- Cada consumidor do componente forma um contrato diferente com base em seus requisitos. Se o componente está sujeito a alterações ao longo do tempo, é importante que os contratos de cada um dos consumidores continuem a ser satisfeitos. Os testes de contrato de integração fornecem um mecanismo para verificar explicitamente que um componente atende a um contrato.



Testes de contrato

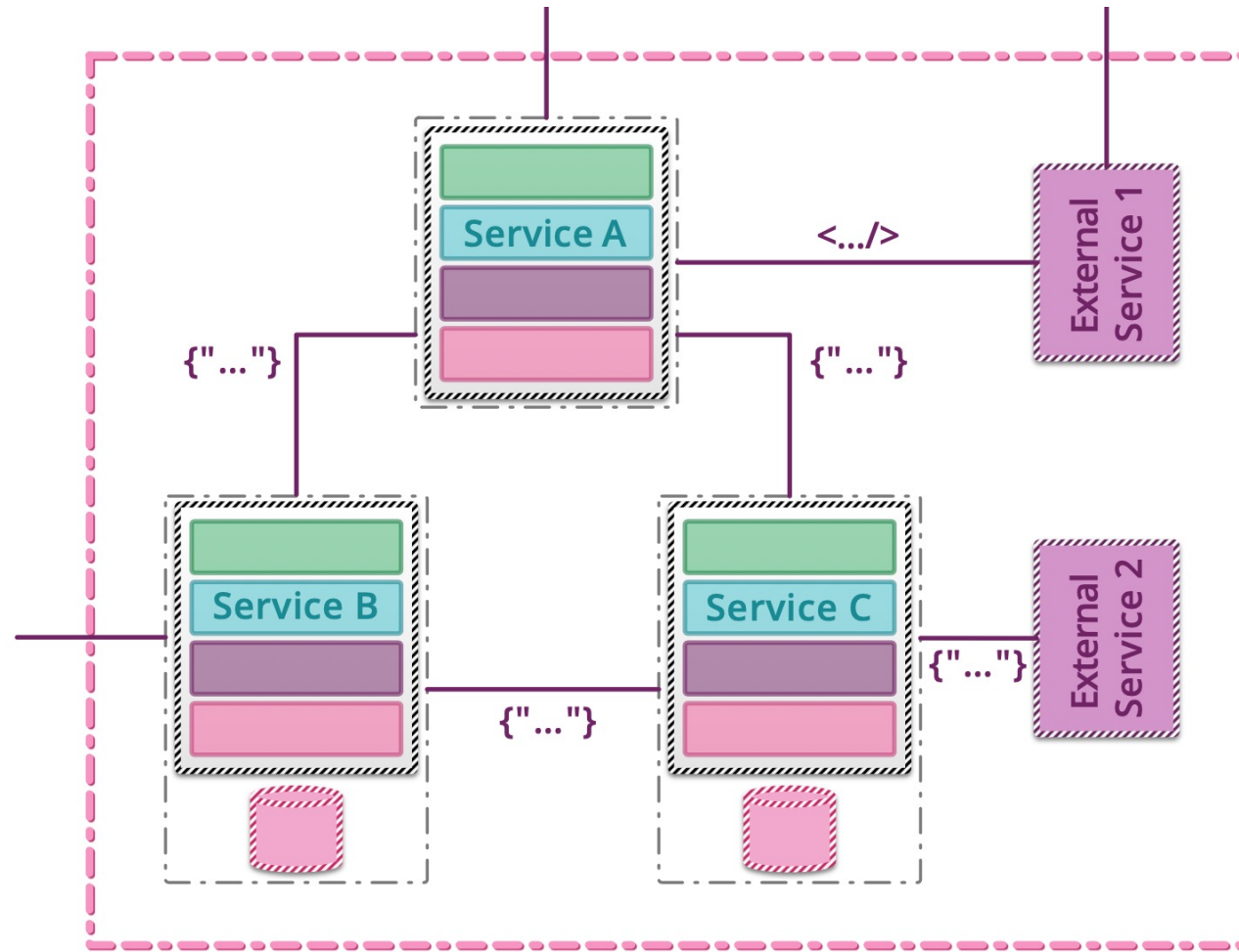


Testes de ponta a ponta (E2E)

- **Um teste de ponta a ponta verifica se um sistema atende aos requisitos externos e alcança seus objetivos, testando todo o sistema, de ponta a ponta.**
- Em contraste com outros tipos de teste, a intenção com testes de ponta a ponta é verificar se o sistema como um todo atende aos objetivos de negócios, independentemente da arquitetura do componente em uso.
- Para conseguir isso, o sistema é tratado como uma caixa preta e os testes exercitam o máximo possível do sistema totalmente implantado, manipulando-o por meio de interfaces públicas, como GUIs e APIs de serviço.
- Como os testes de ponta a ponta são mais voltados para os negócios, eles geralmente utilizam DSLs legíveis para empresas que expressam casos de teste no idioma do domínio com o uso de BDD.



Testes de ponta a ponta



Fonte: <https://martinfowler.com/articles/microservice-testing>



Testes de ponta a ponta

- Escreva o mínimo possível de testes de ponta a ponta
- Foco em personas e jornadas de usuários
- Escolha suas extremidades com sabedoria
- Confie na infraestrutura como código para repetibilidade
- Torne os testes independentes de dados

Resumo

- Considere a pirâmide de testes para estabelecer qualidade contínua e eficiente em seus microsserviços.
- Testes de microsserviços são mais diversos e complexos que testes em arquiteturas tradicionais.
- Reserve tempo apropriado do ciclo de vida do seu produto para o esforço de testes.
- Introduza automação da infraestrutura de testes.





PUC Minas
Virtual